

TMM Telemetry Pipelines

F5 CNF Stats Aggregator and tmmscope — a side-by-side reference

Neutral comparison · 2026-06-23 · both systems collect F5 TMM statistics; they make different design choices. This document states those choices factually and does not rank them. Items marked **not documented** were not found in F5's public docs as of this date.

Both pipelines surface TMM telemetry in Prometheus/Grafana. The **F5 CNF Stats Aggregator** is an in-cluster, operator-managed service that collects, persists, and aggregates TMM statistics. **tmmscope** is a self-contained CLI that injects a per-pod sidecar and pushes raw samples to a host-local stack. The sections below compare them across the dimensions that differ.

1. Architecture and components

Aspect	F5 CNF Stats Aggregator	tmmscope
Components	Operator/Coordinator (Deployment) + Receiver (StatefulSet) + Observer (StatefulSet) + TMM Scraper container in each TMM pod.	One sidecar container per TMM pod + a host-side docker compose stack (Prometheus + Grafana). No operator.
Lifecycle	Operator orchestrates the collect/aggregate lifecycle.	CLI inject/eject (direct patch or admission webhook); re-injection is a no-op.
Relationship	Part of the CNF stats product; runs the native observer/OTEL pipeline.	Additive — runs alongside the native pipeline, does not modify or replace it.

2. Collection model and data path

Aspect	F5 CNF Stats Aggregator	tmmscope
Collection	Pull: the Receiver requests, the in-pod TMM Scraper streams <code>tmctl</code> metrics over gRPC.	The sidecar reads the <code>tmstat</code> shared-memory segment in-pod and pushes outward.
Path to Prometheus	Scraper → Receiver → Observer → OTEL collector; Prometheus scrapes the collector.	Sidecar POSTs directly to Prometheus remote_write ; no intermediate tier.
Storage tier	Embedded LSM time-series DB (memtables → SSTables, compaction; read/write amplification), persisted on the Receiver.	None on the producer side; samples land directly in the host Prometheus TSDB (off-cluster).

3. Wire protocol, format, and transport security

Aspect	F5 CNF Stats Aggregator	tmmscope
Direction	Prometheus pulls (scrape).	Sidecar pushes (remote_write).
Format	Prometheus/OpenMetrics text exposition (OTLP internally between observer and collector).	Prometheus remote_write protobuf, snappy-compressed.

Transport security	HTTPS + mTLS on the scrape; gRPC + mTLS on the internal hops. Encrypted end to end.	Plain HTTP on the data path (<code>http://<host-gateway>:9491/api/v1/write</code>); TLS is used only for the admission webhook (control plane).
Bandwidth drivers	Coarser default cadence and aggregated cardinality reduce the Prometheus-facing volume; internal gRPC streams and DB compaction carry traffic that is not visible on the scrape link.	Compact per-sample encoding, but a fixed 2 s cadence and per-pod cardinality; no internal tier behind it.

Absolute bandwidth depends on pod count, series cardinality, and configured intervals; no fixed figures are published for either system.

4. Time resolution and reporting semantics

Aspect	F5 CNF Stats Aggregator	tmmscope
Cadence	Per-table; default <code>default_collect_interval: 60s</code> / per-table default 1 min. The smallest interval shown in the docs is 3s (diagnostic example). No hard minimum is stated.	Fixed <code>TMSTAT_PUSH_INTERVAL</code> , default 2s; single knob.
Sampling nature	Point sample per tick.	Point sample per tick.
Statistic per metric	By data type: monotonic counters as cumulative sums; gauges as the instantaneous value at the tick. No peak/average-over-window rollup.	Same model; counters cumulative, gauges instantaneous. CPU% is derived from cycle counters rather than TMM's pre-averaged <code>cpu_usage_*</code> fields.

Both are point samplers: a spike shorter than the configured tick is smoothed in either system.

5. Aggregation and per-TMM visibility

The systems differ in **where** cross-TMM aggregation happens, which affects how a single misbehaving TMM (traffic or request-rate spike) becomes visible.

Aspect	F5 CNF Stats Aggregator	tmmscope
Where aggregation happens	At collection time, per table, in one of three modes: Aggregated (combine all TMMs), Semi-Aggregated (combine, exclude dead TMMs), Diagnostic (no aggregation; raw per-TMM).	Not performed by the pipeline. Every series is raw per-TMM (keyed by <code>pod/node</code>); aggregation is done at query time in PromQL.
Default view	Aggregated at 60 s — a cluster total.	Per-TMM at 2 s — individual series.
Seeing one TMM	Enable a Diagnostic -mode job for that table (multiple jobs per table are allowed, e.g. aggregated + diagnostic in parallel).	Always available; <code>sum()</code> at query time when a cluster view is wanted instead.

Diagnostic-mode caveats (F5 CNF Stats Aggregator)

Per-TMM visibility in the aggregator is available through Diagnostic mode, subject to the following:

1. **Opt-in, per table, ahead of time.** The default is Aggregated@60s, which does not retain per-TMM detail. Aggregation occurs at ingest, so data already merged **cannot be de-aggregated later** — a Diagnostic job must be configured for the table *before* the event of interest.
2. **Semi-Aggregated does not help.** It still merges across TMMs; only Diagnostic mode preserves per-TMM identity, so a single-TMM spike is otherwise absorbed into the combined value.
3. **Added cost.** Diagnostic mode multiplies cardinality (per-TMM series) and is typically run at a short interval, increasing bandwidth and storage; it is usually enabled selectively rather than for every table.
4. **Still point-sampled.** Even at a 3 s diagnostic interval, a sub-interval burst is smoothed; the 60 s default would not show it at all.
5. **Configuration surface.** Modes and intervals are set per table in the operator ConfigMap (e.g. jobs: [{table, mode, interval}]), requiring an operator-side change to adjust visibility.

tmmscope does not have these caveats because it performs no ingest-time aggregation; the corresponding trade-offs on its side are the plain-HTTP data path, the off-cluster Prometheus stack, and aggregation deferred to query time.

6. Resource footprint and node placement

Aspect	F5 CNF Stats Aggregator	tmmscope
Standing footprint	Operator + two StatefulSets + OTEL collector + mTLS certs; PersistentVolumes for the metricsdb; ongoing compaction CPU/IO.	Per-pod sidecar only: requests/limits cpu 50m, memory 256Mi (direct patch) or req 25m/32Mi, limit 50m/64Mi (webhook). No PV, no operator, no DB in-cluster.
In-pod / on-DPU part	TMM Scraper container inside each TMM pod (on the DPU's Arm cores), replacing tmstatsd.	Sidecar inside each TMM pod (on the DPU's Arm cores); reads shared memory, pushes off-cluster.
Aggregation/ storage tier placement	Receiver/Observer scheduling is not documented — no nodeSelector, affinity, tolerations, replica counts, or PVC sizing published. By scheduler default these land on host worker nodes, but nothing in the published config pins them off the DPU.	No such tier exists; all storage/query lives in the host-side stack, outside the cluster.

BIG-IP Next for Kubernetes on BlueField-3 runs one DPU per chassis; only the in-pod scraper/sidecar is on the DPU by construction. Placement of the aggregator StatefulSets is left to the installer/scheduler in the public docs.

7. Durability and retention

Aspect	F5 CNF Stats Aggregator	tmmscope
Where history lives	Persisted in the metricsdb (Receiver) and in Prometheus after scrape.	Persisted in the host Prometheus TSDB for its retention period.

TMM restart	Aggregates can carry a terminated TMM's contribution forward (dead-pod preservation in Aggregated/Semi modes).	The restarted pod's series pauses during the gap; prior history remains; counter resets are handled by PromQL <code>rate()</code> .
Buffering on outage	Receiver persistence buffers ahead of export.	No on-disk queue on the producer; samples during an endpoint outage are not retried.

8. Provided Grafana dashboards

Each project ships a Grafana dashboard. They monitor different subjects and share no metrics: the F5 dashboard observes the aggregator infrastructure itself, while the tmmscope dashboard observes TMM dataplane traffic.

Attribute	observer-dash.json (F5)	tmm-realtime.json (tmmscope)
Title / uid	"Observer Pods — Comprehensive Monitoring" / <code>observer-comprehensive-v1</code>	"TMM Real-Time" / <code>tmm-realtime</code>
Subject	Observer/aggregator pods: gRPC calls, Go runtime, metricsdb storage and aggregation.	TMM dataplane: CPU, throughput, connections, interfaces, pool members, virtual servers.
Metric families	<code>f5_grpc_*</code> , <code>go_*</code> , <code>f5_metricsdb_*</code> , <code>f5_aggregation_*</code>	<code>f5tmm_*</code>
Size	27 panels + 3 rows; 9 template variables; schemaVersion 42.	6 panels; 1 template variable (<code>cluster</code>); schemaVersion 39.
Datasource wiring	Mixed across panels (see observations below).	Single pinned provisioned datasource (<code>uid: tmm-telemetry</code>) on every panel.
Shared metrics	None — the two dashboards cover disjoint metric sets.	

Observations — observer-dash.json

Findings a dashboard reviewer would flag, in roughly decreasing order of functional impact:

- Inconsistent datasource references.** Four forms coexist: most targets use `{"uid": "$datasource"}` (the datasource variable); panels 4011–4014 and 4017 carry `datasource: null`, which resolves to Grafana's *default* datasource rather than the variable; panel 4015 is bound to the import input `${DS_PROMETHEUS}`; panels 4016/4018 use a fourth spelling. As a result six panels do not follow the datasource picker.
- Counter shown without rate().** Panel 4018 ("Merged Count Rate") plots the `f5_metricsdb_merged_count_total` counter directly in a gauge, displaying a cumulative total rather than a per-second rate.
- Unscoped template query.** The `table` variable uses `label_values(table)` with no metric selector, enumerating that label across the entire TSDB.
- Variable sourced from an unplotted metric.** The `method` variable queries `f5_grpc_clientside_call_bytes_in_sum`, while every panel uses the `*_call_duration_seconds_*` family.
- Mixed rate windows.** The gRPC panels hardcode `[5m]`; the Go-runtime panels use `$__rate_interval`.

Minor: trailing whitespace in one expression; panel 4013 combines a ratio (read/write amplification) and a count (SSTables) on a single gauge with shared units.

The tmmscope dashboard (`tmm-realtime.json`) pins one provisioned datasource (`uid: tmm-telemetry`) across all six panels, so the datasource-consistency class of finding does not arise; it is a deliberately minimal, curated set driven by a single `cluster` variable. Its narrower scope also means it carries none of the aggregator-infrastructure panels, and vice versa — the two are complementary views rather than alternatives.

9. Metric naming and coverage

The two dashboards share no metrics, but that is a property of the dashboards, not the data. tmmscope reads the same `tmstat` tables that F5's metric catalog (`cnf-otel-stats`) is built from, so the overlap is best described at three levels: source, naming, and scope.

Level	Overlap with F5 <code>cnf-otel-stats</code>	Detail
Source data	High	tmmscope's four tables (<code>tmm_stat</code> , <code>virtual_server_stat</code> , <code>pool_member_stat</code> , <code>interface_stat</code>) are all categories in F5's catalog; same <code>tmstat</code> origin, same underlying values.
Metric names	None	Three separate naming generations (table below); not interchangeable without relabeling.
Coverage	Subset	tmmscope exports 4 tables (L4 traffic, connections, CPU cycles, interfaces). <code>cnf-otel-stats</code> spans 40+ tables (DNS, FW/NAT/CGNAT, DOS, TCP/HTTP/FTP/RTSP profiles, VLANs, ICMP, protocol inspection, ...).

What tmmscope collects is also wider than what it plots: the exporter emits every numeric column of each configured table as `f5tmm_<table>_<column>` (hundreds of fields), while the shipped dashboard surfaces about seven of them. The remaining series are present in Prometheus and available for additional panels.

Same source field, three names

<code>tmstat</code> source field	tmmscope	F5 v1 (raw)	F5 v2 (standardized)
<code>virtual_server_stat · clientside.bytes_in</code>	<code>f5tmm_virtual_server_clientside_bytes_in</code>	<code>virtual_server_stat/[name]/clientside.bytes_in</code>	<code>f5.virtual_server.clientside.received.bytes</code>
<code>pool_member_stat · serverside.pkts_out</code>	<code>f5tmm_pool_member_serverside_pkts_out</code>	<code>pool_member_stat/.../serverside.pkts_out</code>	<code>f5.pool_member.serverside.transmitted.packets</code>
<code>tmm_stat · tm_idle_cycles</code>	<code>f5tmm_tmm_tm_idle_cycles</code>	<code>tmm_stat/tm_idle_cycles</code>	TMM core stats

The same source value appears in every row; only the metric string differs. tmmscope's `f5tmm_*` names are distinct from both the F5 v1 raw table paths and the v2 `f5.*` namespace.

Aspect	F5 <code>cnf-otel-stats</code> catalog	tmmscope
Type metadata	OTLP semantics assigned per metric (Cumulative / Maximum / Minimum / Instance).	All series emitted as <code>gauge</code> , including monotonic counters; counter behaviour is recovered with PromQL <code>rate()</code> .
Surface filtering	Full, unfiltered metric surface.	tmm's internal control-plane objects (leading- <code>_</code> pools/virtuals, link-local addresses) are dropped before export.

Net: against the dataplane catalog the two overlap substantially in substance (same four `tmstat` tables, same field values) but not in metric names (a distinct `f5tmm_*` generation) and not in breadth (a curated subset of a 40+ table catalog).

10. F5 v1 vs v2 vs tmmscope

F5 exposes TMM stats in two generations. Sections 1–7 above describe the **v2** (aggregated) path; **v1** is an older, non-aggregated path that is in some respects closer to tmmscope. The key contrast is how each reaches Prometheus.

How v1 streams to Prometheus: in v1, `tmstatsd` inside TMM streams metrics (OTLP) directly to the OTEL collector — *without aggregation* — and Prometheus scrapes that collector. v1 **bypasses the Receiver/Observer aggregator entirely**; v2 instead routes the in-pod TMM Scraper through Receiver → Observer (where aggregation happens) before the same OTEL collector. Both v1 and v2 are therefore push-to-OTEL then pull-by-Prometheus; tmmscope skips OTEL and pushes `remote_write` straight to Prometheus.

Aspect	F5 v1 (raw)	F5 v2 (aggregated)	tmmscope
Collection from TMM	<code>tmstatsd</code> streams from TMM.	In-pod TMM Scraper (replaces <code>tmstatsd</code>) streams <code>tmctl</code> to the Receiver on request, gRPC + mTLS.	Sidecar reads the <code>tmstat</code> shared-memory segment in-pod.
Collection cadence (per TMM)	Not stated in the docs; <code>tmstatsd</code> streams from each TMM periodically.	Per-table; default 60s, as low as 3s shown (no hard minimum). Applied before/at aggregation.	Fixed 2s push interval, identical for every TMM.
Aggregation tier	None — bypasses Receiver/Observer.	Receiver + Observer StatefulSets.	None.
Aggregation	None; raw per-TMM.	Aggregated / Semi-Aggregated / Diagnostic (per table).	None; aggregation done at query time in PromQL.
Path to Prometheus	<code>tmstatsd</code> → OTEL collector → Prometheus scrape.	Scraper → Receiver → Observer → OTEL collector → Prometheus scrape.	Sidecar → <code>remote_write</code> → Prometheus (no OTEL).
Prometheus ingestion	Pull (scrape of the OTEL collector).	Pull (scrape of the OTEL collector).	Push (<code>remote_write</code>).
Transport security	HTTPS + mTLS scrape of the OTEL collector.	HTTPS + mTLS scrape; gRPC + mTLS on the internal hops.	Plain HTTP on the data path.
Metric naming	Raw table-based, e.g. <code>virtual_server_stat/[name]/clientside.bytes_out</code> .	Standardized fixed names, e.g. <code>f5.virtual_server.clientside.received.bytes</code> .	<code>f5tmm_<table>_<column></code> , e.g. <code>f5tmm_virtual_server_clientside_bytes_out</code> .
Identity labels	<code>host.name</code> , <code>tmmID</code> (per-TMM instance).	namespace, target namespace, <code>observer.job.mode</code> (aggregation state).	pod / node + the table's key columns.
Per-TMM granularity	Always (raw per-TMM).	Only in Diagnostic mode; otherwise merged.	Always.

In granularity and the absence of an aggregation tier, v1 is the closest F5 analog to tmmscope; they differ mainly in the egress (v1 goes through OTEL and is scraped over mTLS; tmmscope pushes `remote_write` over plain HTTP) and in naming. v2 adds the aggregation tier and standardized `f5.*` names on top of the same TMM source data. The page does not restate the Prometheus scrape step for v1 specifically, but the OTEL collector is the documented Prometheus integration point for both generations.

Summary

The two pipelines occupy different points on the same design space. The F5 CNF Stats Aggregator centralizes collection, persistence, aggregation, and mTLS in an operator-managed in-cluster service, and exposes per-TMM detail through opt-in Diagnostic mode. tmmscope keeps a small per-pod sidecar, defers storage and aggregation to an off-cluster Prometheus/Grafana stack, and presents per-TMM series by default over a plain-HTTP push. The right fit depends on whether ingest-time aggregation and durability or always-on per-pod granularity and a minimal in-cluster footprint matter more for a given deployment.